14/05/14

Mobile Shoppe Project

Mobile Shoppe is a windows application which is computerizing mobile shop day to day transaction

This application will be consuming by two types of users

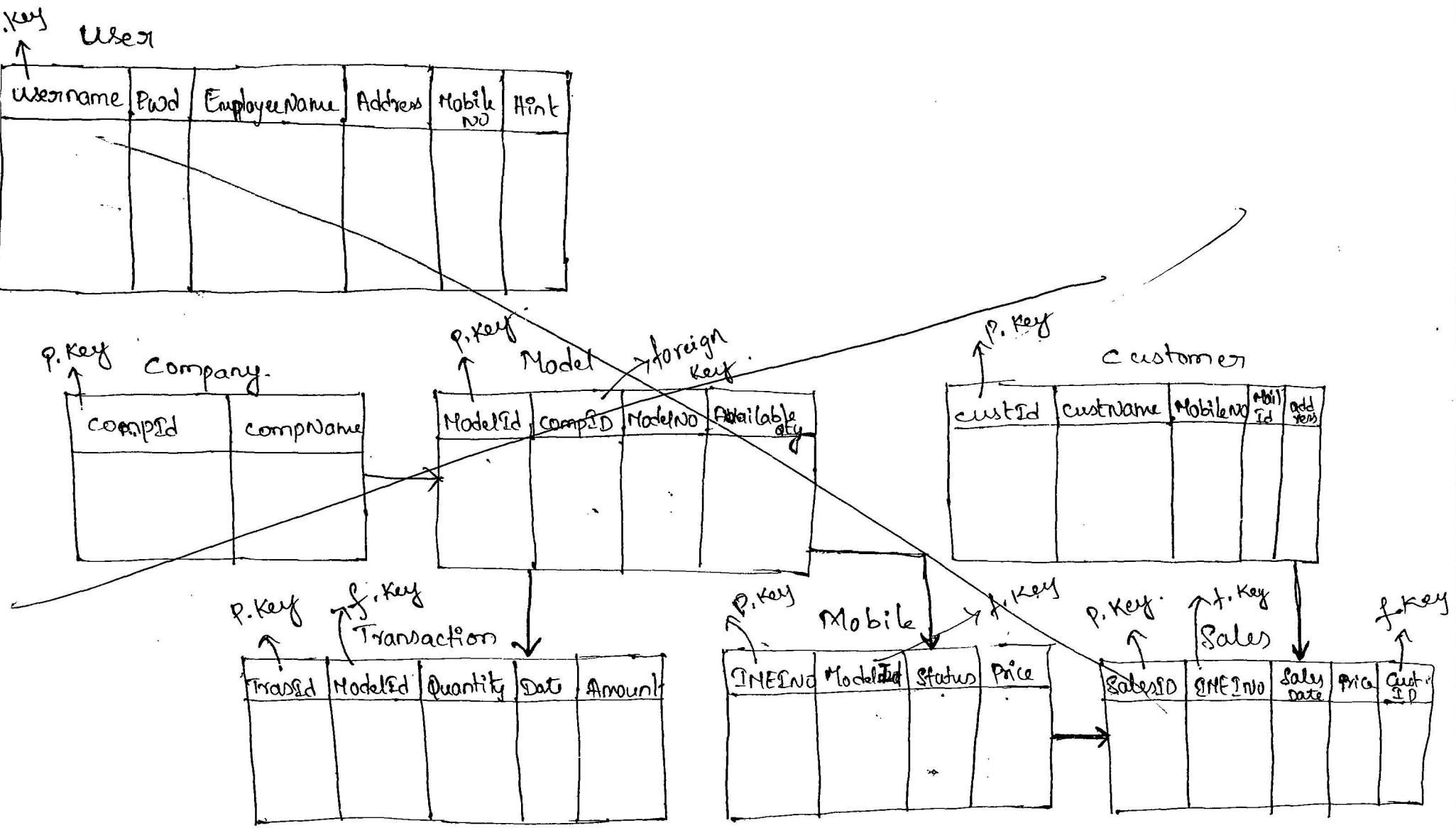
1. Admin (owner of the shop)
2. Employee (Sales men).

Admin Privilages :

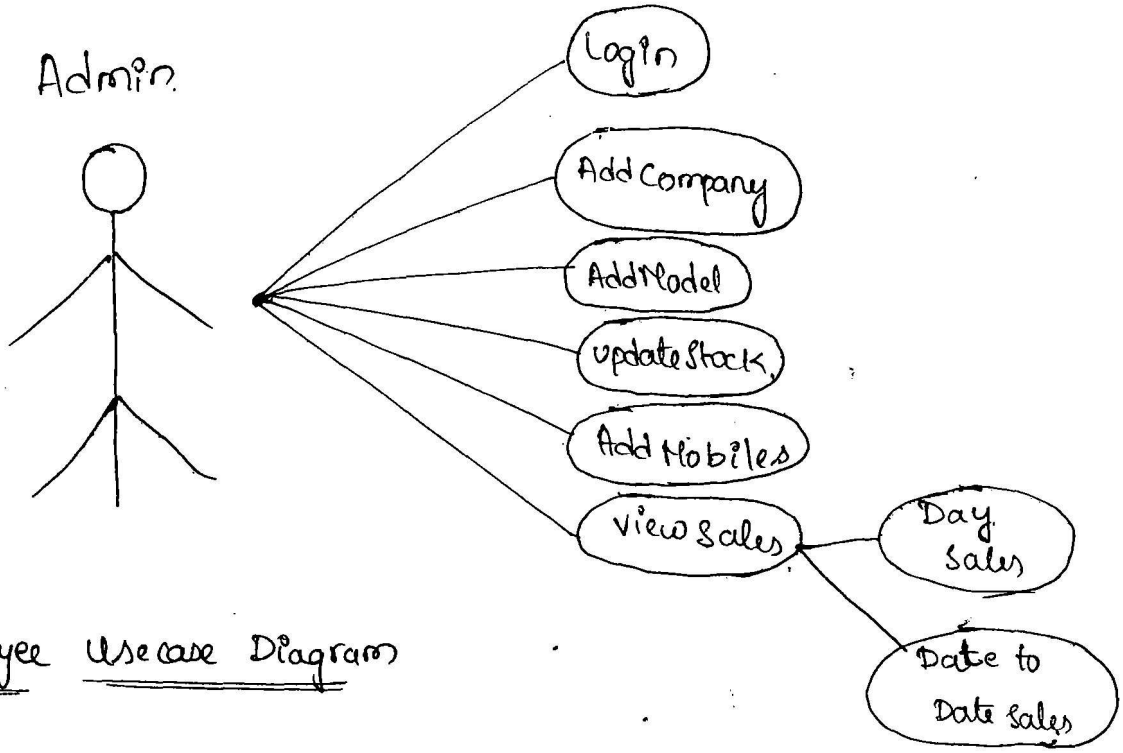
→ Adding company Names.
→ Adding mobile models
→ updating Stock
→ Adding mobiles with IMEI number
→ Adding employees.
→ view Sales details (date wise & date to date)

Employees (Sales men) Privilage :

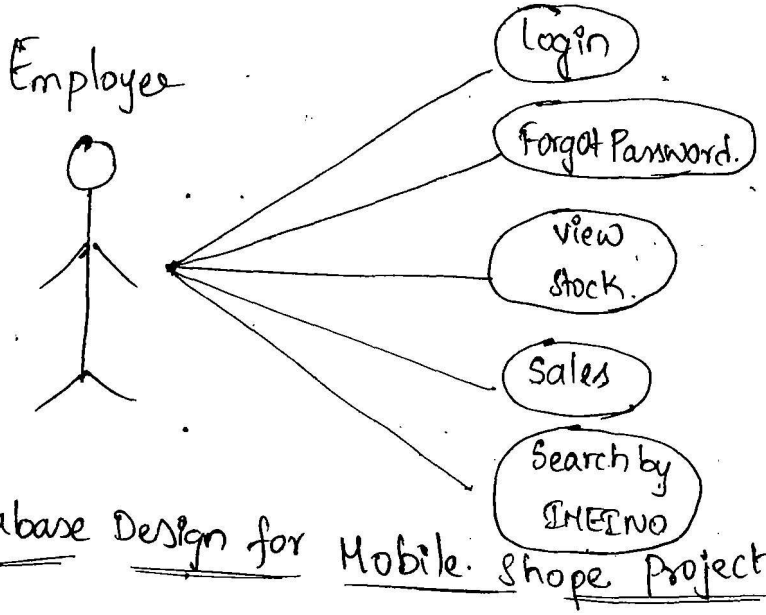
→ login
→ Forgot Password.
→ checking Stock
→ Adding Sales details
→ Searching Customer Information based on IMEI no.


P.Key
User
Username
Pwd
EmployeeName
Address
Mobile NO
Hint
P.Key
Company.
CompId
CompName
P.Key
Model
Foreign Key
ModelId
CompID
ModelNO
Available qty
P.Key
Customer
CustId
CustName
MobileNo
Mail Id
add ress
P.Key
f.Key
Transaction
TransId
ModelId
Quantity
Date
Amount
P.Key
Mobile
f.Key
IMEINO
ModelId
Status
Price
P.Key
f.Key
Sales
f.Key
SalesID
IMEINO
Sales date
Price
Cust ID

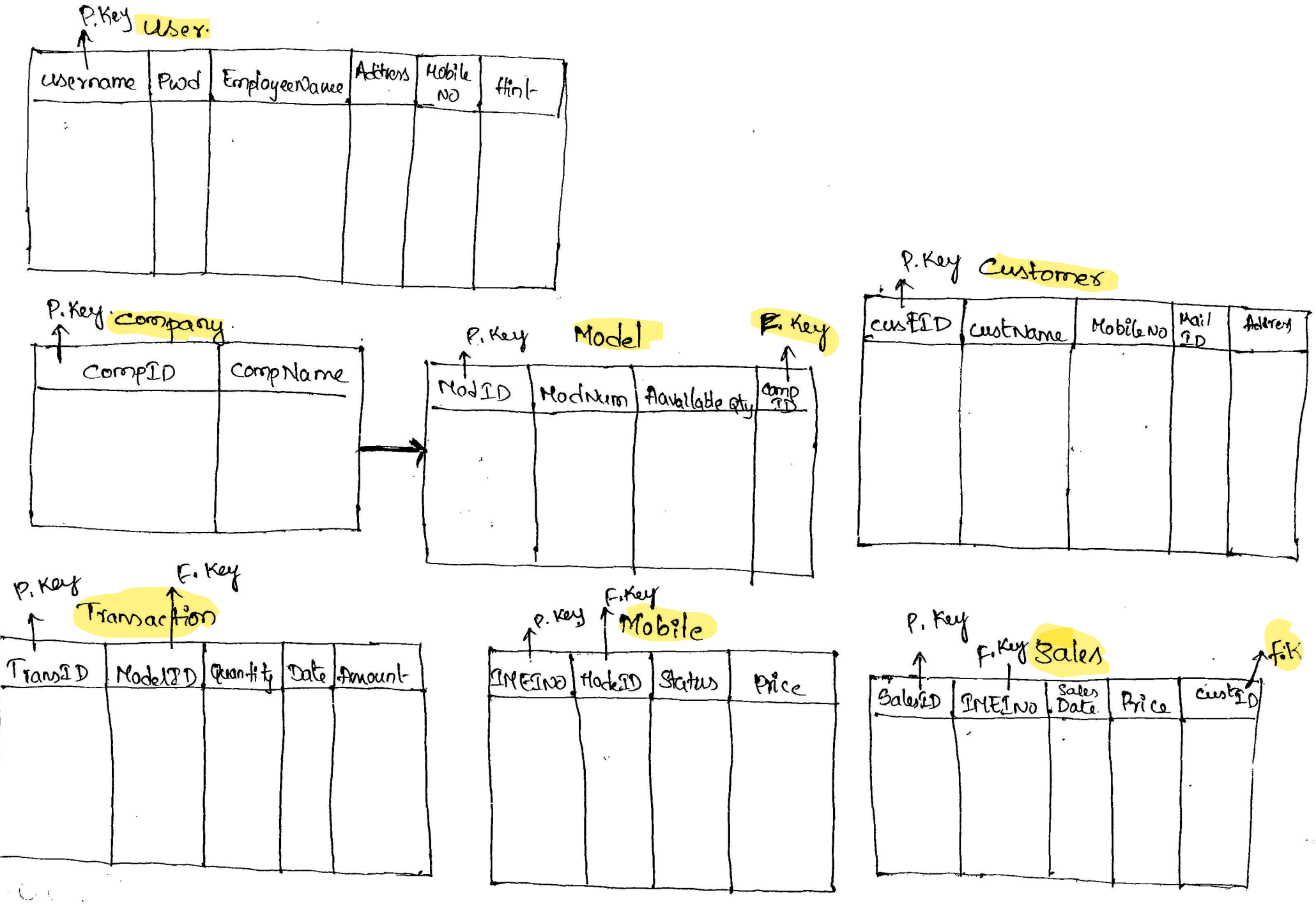
Admin Usecase Diagram



Employee Usecase Diagram

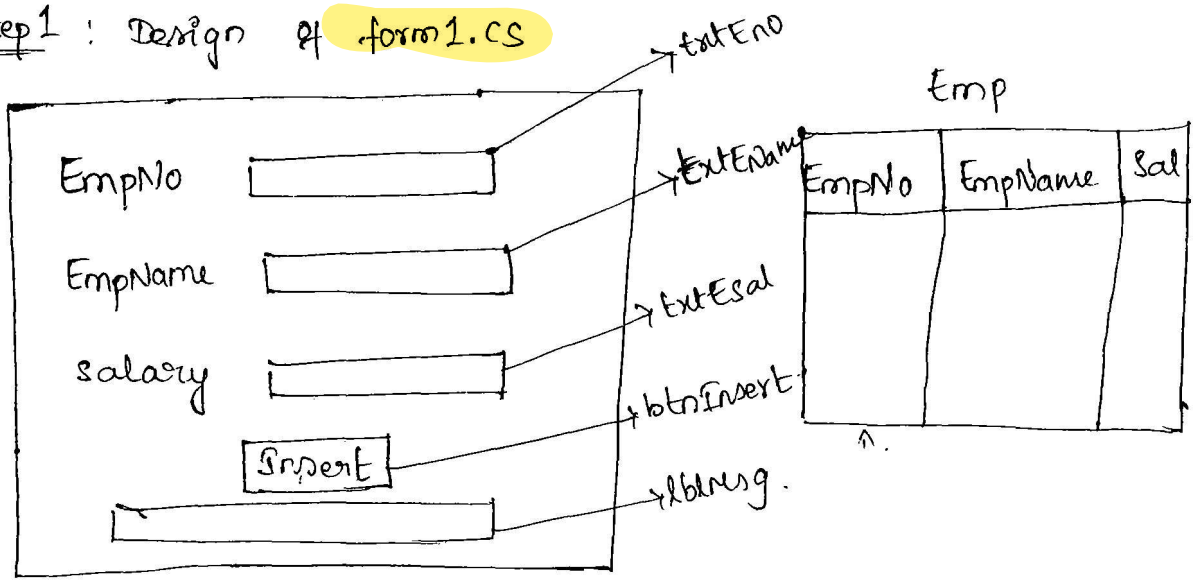


Database Design for Mobile Shope Project


P.Key
User
username
Pwd
EmployeeName
Address
Mobile NO
Hint
P.Key
Company
CompID
CompName
P.Key
Model
F.Key
ModID
ModNum
Available qty
Comp ID
P.Key
Customer
cusEID
custName
MobileNO
Mail ID
Address
P.Key
F.Key
Transaction
TransID
ModelID
Quantity
Date
Amount
P.Key
F.Key
Mobile
IMEINO
ModID
Status
Price
P.Key
F.Key
Sales
f.k
SalesID
IMEINO
Sales Date
Price
custID

Example to create AutogeneratedID by using C#.Net code.

Step 1 : Design of form1.cs




```
using System.Data.SqlClient;
namespace AutogenerateIDEx
{
  class Form1
  {
    void
    SqlConnection conn = new SqlConnection("Server=.; database = mydatabase
    SqlCommand cmd;                                    uid=sa; pwd=abc;");
    internal void AutoGenId()
    {
      cmd = new SqlCommand("Select ISNULL(MAX(EmpNo),0) from Emp", conn);
      conn.Open();
      int i = cmd.ExecuteScalar(); i++;
      conn.Close();
      txtEno.Text = i.ToString();
    }
    void Form_Load()
    {
      AutoGenId();
    }
```

```
void btnInsert_Click( )
{
   int eno = Int.Parse(txtEno.Text);
   String ename = txtEname.Text;
double esal = double.Parse(txtEsal.Text);
   cmd = new SqlCommand("Insert into Emp values(@eno,
                                    @ename, @esal)", conn);
cmd.Parameters.AddWithValue("@eno", eno);
cmd.Parameters.AddWithValue("@ename", ename);
cmd.Parameters.AddWithvalue("@esal", esal);
conn.open();
int i = cmd.ExecuteNonQuery();
conn.close();
AutoGenID();
txtEname.clear();
txtEsal.clear();
txtEname.Focus();
if(i==1)
  { lblMsg.Text = "Record Inserted";
  }
else
  { lblMsg.Text = "Record not inserted";
  }
}
```

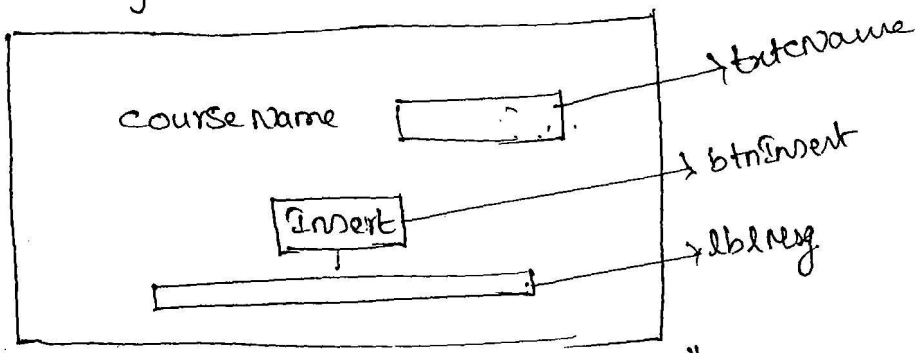
Example to Generate the ID's automatically with the help of identity concept within SqlServer.

```
create table course(cid int Primary key Identity(1,1), cname varchar(20))
insert into course values('dotnet')
insert into course values('java')
```

cid	cname
1	dotnet
2	java

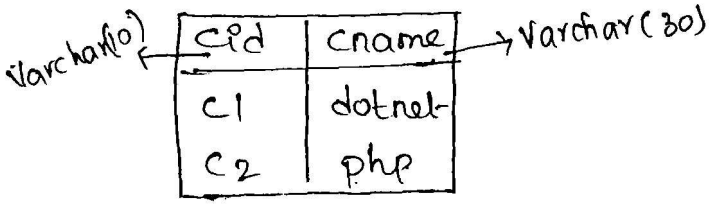
→ Giving course names from front end.

Step 1 Design




```
SqlConnection conn = new SqlConnection("Server=.; database=mydatabase;
                                        Uid=sa; Pwd=abc;");
Void btnInsert_Click()
{
    String cname = txtCname.Text;
    SqlCommand cmd = new SqlCommand("Insert into course
                                     values(@cname)", conn);
    cmd.
    conn.Open();
    int i = cmd.ExecuteNonQuery();
    conn.Close();
    if(i==1)
    {
        lblmsg.Text = "Record inserted";
    }
    else
    {
        lblmsg.Text = "Record not inserted";
    }
}
```

Example to generate AutoId's like below



Implementation of Mobile Shope Project

Step 1 : Create a new windows application rename it as Mobile Shope Project.

Step 2 : Add six windows forms to the Solution Explorer they are

1. UserLogin.cs
2. AdminLogin.cs
3. AdminHomepage.cs
4. UserHomepage.cs
5. ForgotPassword.cs
6. ConfirmDetails.cs

Step 3 : Set userLogin.cs as StartupPage.

Step 4 : Create a database within SQL Server that is "MobileShopedb"

Step 5 : Create 7 tables within database like below d/c to design.

1. tbl_company
2. tbl_Model
3. tbl_Transaction
4. tbl_Mobile
5. tbl_Customer
6. tbl_Sales
7. tbl_user

Step 6 : Declare the GlobalConnectionString within app.config file.

Admin Module

UserLogin.cs

```
class UserLogin
{
  void LinkLabel_LinkClick()
  {
    AdminLogin objLogin = new AdminLogin();
    objLogin.Show();
    this.Hide();
  }
}
```

AdminLogin.cs

```
class AdminLogin
{
  void linkBack_LinkClick()
  {
    UserLogin objLogin = new Uselogin();
    objLogin.Show();
    this.Hide();
  }
  void btnlogin_click()
  {
    if (txtUid.Text == "admin" && txtpwd.Text == "admin")
    {
      AdminHomePage objAdminHome = new AdminHomePage();
      objAdminHome.Show();
      this.Hide();
    }
```

```
else
{
    lblMsg.Text=" User is not valid";
}
}
}
```

AddingCompanies

→ This requirement we can implement in 2 tasks

1. Generating CompanyId automatically & binding generated Id within txtbox.
2. Inserting the record into company table.

Below code in AdminHomePage.cs

```
class AdminHomePage
{
SqlConnection Conn = new SqlConnection(ConfigurationManager.ConnectionStrings
                                        ["cs"].ToString());
SqlCommand cmd;
void AutoGenId()
{
cmd = new SqlCommand("Select ISNULL(MAX(compID),0) from Company",
conn);
Conn.open();
int i = cmd.ExecuteScalar();
i++;
Conn.close();
txtCompID.Text = i.ToString();
}
void AdminHomePage Page_Load()
{
    AutoGenId();
}
```

```
Void btnAdd_Click()
{
    int compID = int.Parse(txtCompID.Text);
        String compName = txtCompname.Text;
cmd = new SqlCommand("Insert into Company values(@compID,
                        @compname)", conn);
cmd.Parameters.Add withvalue("@compID", compID);
cmd.Parameters.Add Withvalue("@compname", compname);
conn.open();
cmd = ExecuteNonQuery();
    AutoGenID();
    conn.close();
    txtcompname.clear();
    }
}
```

Adding Models

```
using System.Data.SqlClient;
using System.Data;
```

Updating Stock

This requirement we can implement by using following tasks

1. Trasaction ID generation automatically.
2. Binding company names to combobox1 ie., ComboCname.
3. Binding the model numbers based on selected company name companyId.
4. Inserting transaction details ie., transactionID, ModelID, Qty, current date, amount into transaction table. and update the model table. availableQty based on modelID

```
class AdminHomePage
{
   Internal void AutoTransID()
   {
    // write the code to generate transactionId automatically & assign to txtTransId control.
   }
   internal void BindingCompanyNames()
    {
     // same as above
    }
  void ComboCname_SelectedIndexChanged()
  {
    int compID = Int.Parse(txtcombocname.Text);
    Cmd.Bas cmd = new SqlCommand("Select mid, Mobnum,-
                                  where. compid = @cid",com);
    conn.open(); c
    cmd.ExecuteScalar();
```

internal void btnUpdate_click()
{

Adding Mobiles

1. Binding Company Names to combobox.
2. Binding Modelnumbers to combobox based on Selected companyname companyId
3. Inserting into mobile table & assign by default status as not sold.

Add Employee

User Module.

below code in userlogin.cs

```
SqlConnection conn = new SqlConnection(con.
SqlCommand cmd;  ①
                 ②
String.  Select count(*). from user where. {username = @①
                                            Password = @②"', conn)
```

below code in forgotPassword.cs.

```
class ·ForgotPassword
// Select Pwd. from tbl_User where. username = @uid. and.
   hint = @hint.

   String. st = txtUname.Text;
   String hint = txthint.Text;
   cmd = new SqlCommand("Select Pwd from tbl_User where
     username = @st and hint = @hint", conn);
// Passing values to parameters
   conn.Open();
   int i = cmd.ExecuteScalar();
   conn.Close();
   if(i=21
```

Sales Module

This requirement we can impement in 2 steps

Step 1: user homepage.cs

1. Binding companynames
2. Binding Model numbers based on selected companyname.
3. Binding IMEINo, Price based on selected modelno whose status is not sold. and price.
4. When user click Submit button redirect to confirmdetails

Step 2 Below code within userhomepage.cs

```
class UserHomePage
{
  //task1

  BindingCompanies()
  {
      // same as above
  }
//task 2
  Binding modelNum()
  {
    // same as above
  }
//task 3
    Binding IMEINOPrice()
    {

      cmd = new SqlCommand(" select * from tbl_
```

```
Voidcombo_SelectedIndexchanged()
{
    double cost = double.Parse(comboIMEINo.Text);
    txtPrice.Text = cost;
}
void btnSubmit_clickl()
{
    ConfirmDetails.objconfirm = new confirmDetails();
    objconfirm.show();
    this.Hide();
}
```

assign customer name, Company name, modelno, mobileno, address, IMEIno, EmailId, price from user home page.cs controls to confirm details.cs label controls.

Step 2 : Confirm details (confirmdetails.cs)

1. Autogen CustId, autogenSalesId.
2. Inserting Customer details into customer table & Sales details into Sales table.
3. Updating Status column of mobile table as Sold based on IMEINo and update available qty in model table. by reducing 1 based on modelID
4. When you click cancel button hide confirmdetails

below code within confirm details.cs

```
class Confirmdetails
{
    // 1
    autocustID()
    {
    }
    // 2
    autoSalesID()
    {
    }
```

```
void btnOK_click()
{
    int    custid = int.Parse(?.Text);
    String custname = lblcustname.Text;
    long   Mobilenum = long.Parse(lblMobilenum.Text);
    String custmailId = lblmailId.Text;
    String Address = lblAdress.Text;
    cmd = new SqlCommand("insert
    int    Salesid = int.Parse(?.Text);
    long InvoiceNo = Long.Parse
```

Below code within userhomePage.cs

```
class userhomePage
{
    // task 1
    Binding company
    {
    }
    // task 2
      Binding ModelNo()
      {
      }
    user Homepage_load()
    {
        Bindingcompany();
        {
        }
    }
```

```
void comboModelNum_SelectedIndexChanged()
{ mid - - -
  cmd = new SqlCommand("Select AvailableQty. from tbl_Model where
                                                    ModelId = @mid", conn);
  cmd.Parameters.AddWithValue("@mid", mid);
  conn.open();
SqlDataReader dr = cmd.ExecuteReader()
  DataSet ds = DataTable dt = new DataTable()
```

Searching customer based on IMEINO

write the below code within userhomepage.cs.

```
class UserHomePage
{
   void linkSearch_Click()
    {
      long ImeiNo = long.Parse(txtIMEINo.Text);
  SqlCommand cmd = new SqlCommand(" Select c1.CustomerName,
          c1.CustMobileNumber, C1.EmailId, c1.Address from tbl_Sales S1
          Inner Join tbl_Customer c1 on S1.CustId = c1.CustId
          where S1.IMEINO = @ImeiNo)", Conn);
  cmd.Parameters.AddWithValue("@ImeiNo", ImeiNo);
  conn.open();
  SqlDataReader dr = cmd.ExecuteReader();
   GridCustomer.DataSource = dr;
    conn.close();
  }
}
```

Sales Report (Admin)

Write a Query to get Sales details based on single date.

```
Select  S1.SalesId, C1.CompanyName, Md1.ModelNum, S1.IMEINO,
    S1.Price from tbl_Sales S1 Inner join tbl_Mobile Mb1
    on  S1.IMEINo = Mb1.IMEINO inner join tbl_Model
    Md1  on  Mb1.ModelId = Md1.ModelId Inner join
    tbl_Company C1 on Md1.CompId = C1.CompId where.
    S1.IMEINO = @IMeiNO.   S1.date = @d.
```

```
class  AdMinHomePage
{
    void DateTimePicker_ValueChanged()
    {
        DateTime Date = Convert.ToDateTime(DateTimePicker1.Text);

        Cmd = new SqlCommand("Select S1.SalesId, C1.ComaryName,
            Md1.ModelNum, S1.IMEINO, S1.Price from tbl_Sales S1
            Inner join tbl_Mobile.Mb1 on S1.IMEINo = Mb1.IMEINo
            inner join tbl_Model.Md1 on Mb1.ModelId = Md1.ModelId.
            inner join tbl_company C1 on Md1.compId = C1.CompId
            where S1.date = @date
```

Validations for Mobile Shope.

→ Textboxes should not be empty.

Adding companies

→ Company textbox should not allow duplicate values.

Adding Models

→ Company name should be selected

→ Model number should not be duplicated.

Update Stock

→ Company name, Model Number should be selected

→ Quantity, Amount should allow only numerical values

Adding Mobile

→ Company name, ModelNO should be selected

→ IMEINO, Price should allow only numerical values.

→ IMEINO should not allow duplicate values.

Sales Report

Day → Dont allow the user to select future date

→ When user click Sunday click holiday msg.

→ Let us assume our client has established business on jan 2013, avoid user to select previous data of estabishment date.

Date to Date

→ User Should select valid date for starting & ending date.

→ Starting date should not be before establishing date.

→ ending date should not exceed the current date.

Add Employee

→ Employee user name should not be duplicate

→ Mobile no should not allow other than digits

→ Mobile no's should be (min 10 & max 12)

→ username should have atleast 6 chars, atleast 1 char & 1 number.

→ Password should have 8 chars & Start with capital letter.

→ password should have 1 digit & 1 Spl char.

→ Both passwords should match.

→ Hint should be min 5 chars.

User login and forgot Password

→ Textboxes should not be empty

Sales

→ Mobile no should be digits & should have 12 digits

→ customer name should be string.

→ Address should be string.

→ Email ID (Expression), should be valid

View Stock

→ Should Select company name & model number.

Search customer

→ IMEINO txtbox Should not be empty

→ IMEINO should have 16 digits.

SATHYA TECHNOLOGIES

MOBILE SHOPPE PROJECT

(WINDOWS FORMS(C#.NET))

Address:

2nd Floor, Sri Sai Arcade,

Beside: Adithya Trade Center,

Ameerpet, Hyderabad-500038.

Ph:040-65538958/65538968

SATHYA TECHNOLOGIES-Mobile Shoppe Project Scenario

1.	Name of the Project	MOBILE SHOPPE
2.	Objective/ Vision	Mobile Shoppe is a windows application which is computerizing a mobile shop's day to day transactions. The objective of this project is to maintain the details of the mobile store, like maintaining product details available in the store.
3.	Users of the System	1. Administrator 2. Employee
4.	Functional Requirements	• Administrator can manage the company and model details • He can view the Sales details, Add the Employees • Employee can view the Stock Details and make the sales • Employee can recover the password through Hint Question • Employee can search the customer • Only Authenticated users can access the system
5.	Reports	• Day Wise Reports • Weekly Reports • Monthly Reports
6.	Languages and Technologies to be used	• C#.Net • ADO.NET
7.	Backend	• MS SqlServer-2008
8.	Tools to be Used	• Microsoft Visual Studio Editor-2010 • .NET Framework-4.0 • Unified Modeling Language

MOBILE SHOPPE

Description:

Mobile Shoppe is a windows application which is computerizing a mobile shop's day to day transactions. The objective of this project is to maintain the details of the mobile store, like maintaining product details available in the store.

Admin:

Admin will have following privileges.

- Adding Company Names
- Adding Mobile Models
- Updating Stock
- Adding Mobile Details with IMEI number
- Adding Employees
- Viewing Sales Details(Date Wise and Date to Date)

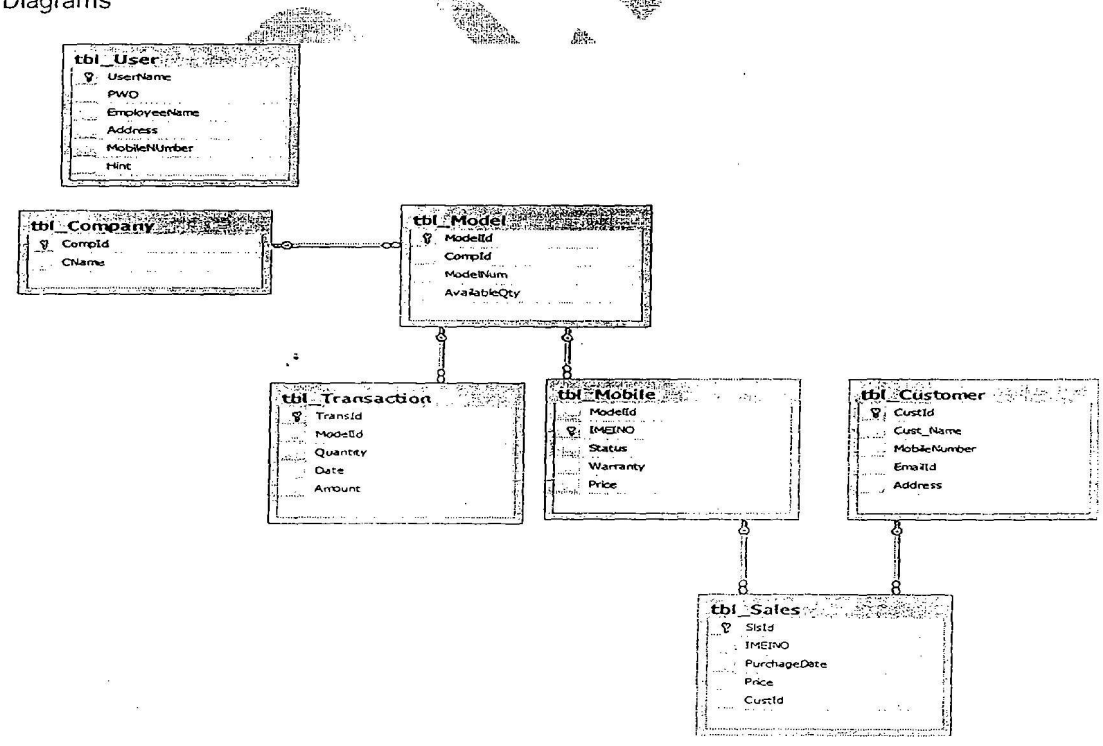
Employee:

Employee will have following privileges.

- Adding Sales Details
- Viewing Stock Details
- Searching Customer Information by using mobile IMEI number

Database Design

ER Diagrams



UseCase Diagrams:

Use case is used to define the core elements and process that make up a system.

It contains two things those are

1. Actor
2. UseCase

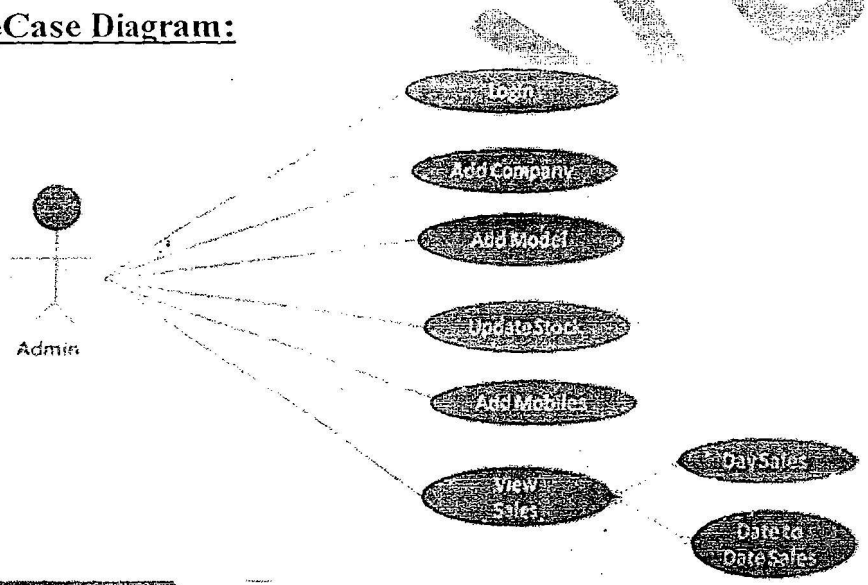
Actor: The actors are also called as key elements.

UseCase: The Use cases are also called as Processes.

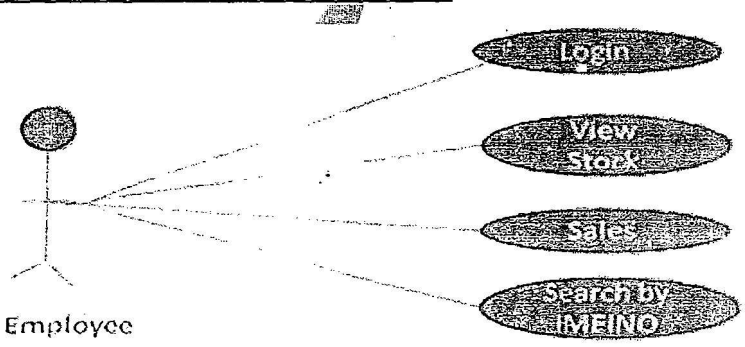
It always gives the relationship between actor and usecases or activities.

It is used for understanding the behavior of the application. It gives the clarity about project development code.

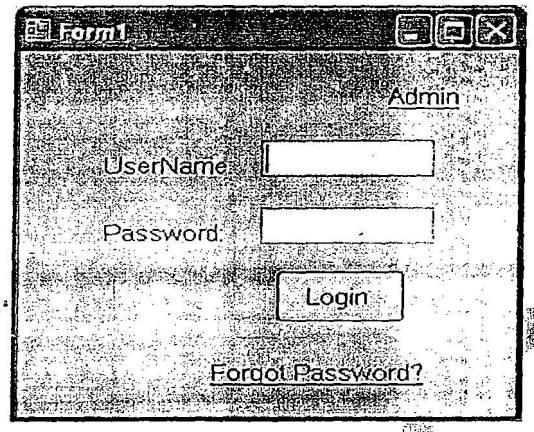
Admin UseCase Diagram:



Employee Use Case Diagram:

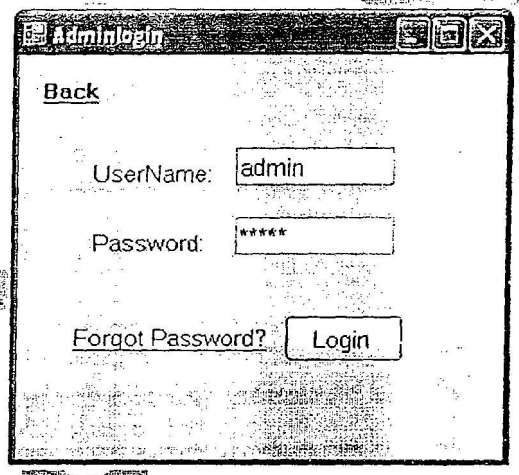


Home Form



- when the user clicks on the admin button it should redirect to "Admin Login" Form

1. **Admin Login Form**



- Give the username and password as "admin".
- When the admin clicks on login button if it is valid it should redirect to "Admin Home" form otherwise it should display an error message.

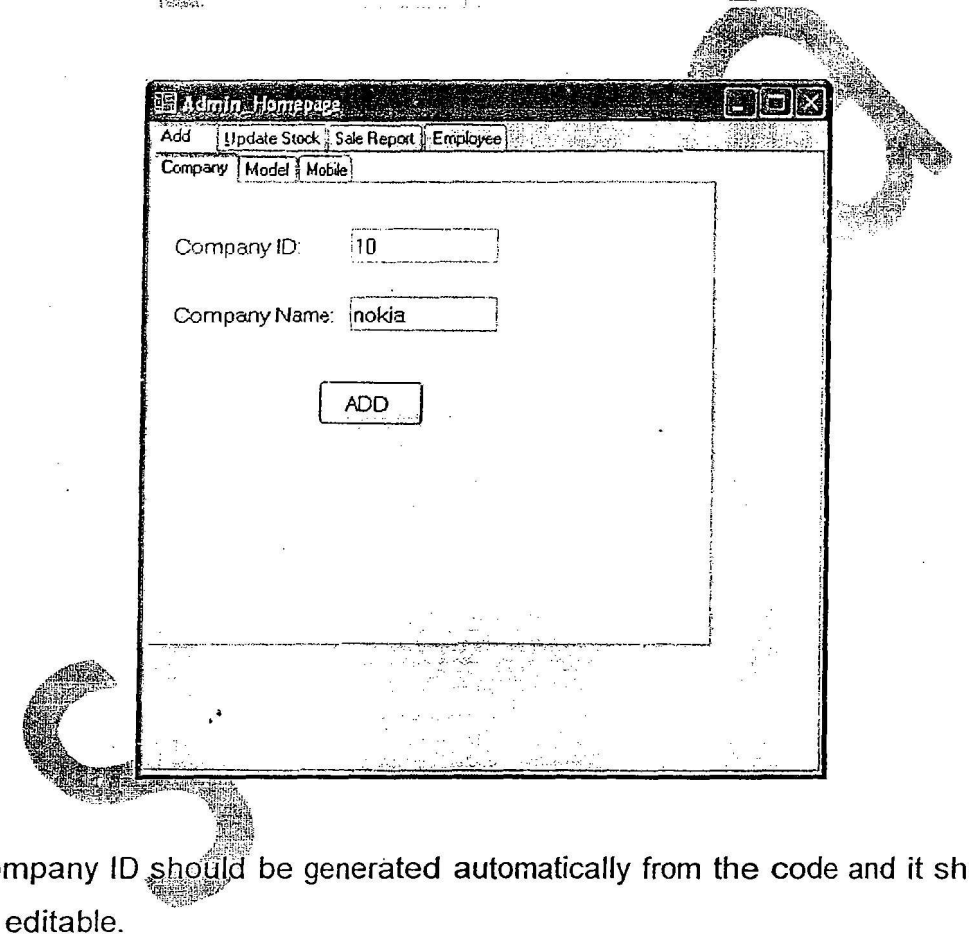
2. Admin Homepage

AddComapany:

Create a table with name "tbl_Company" as shown below.

tbl_Company

Column Name	Data Type	Allow Nulls
CompId	varchar(20)	☐
CName	varchar(20)	☑
		☐



- Company ID should be generated automatically from the code and it should not be editable.
- When the user enters company name and click on add button it must store in to "tbl_Company" table in the database.

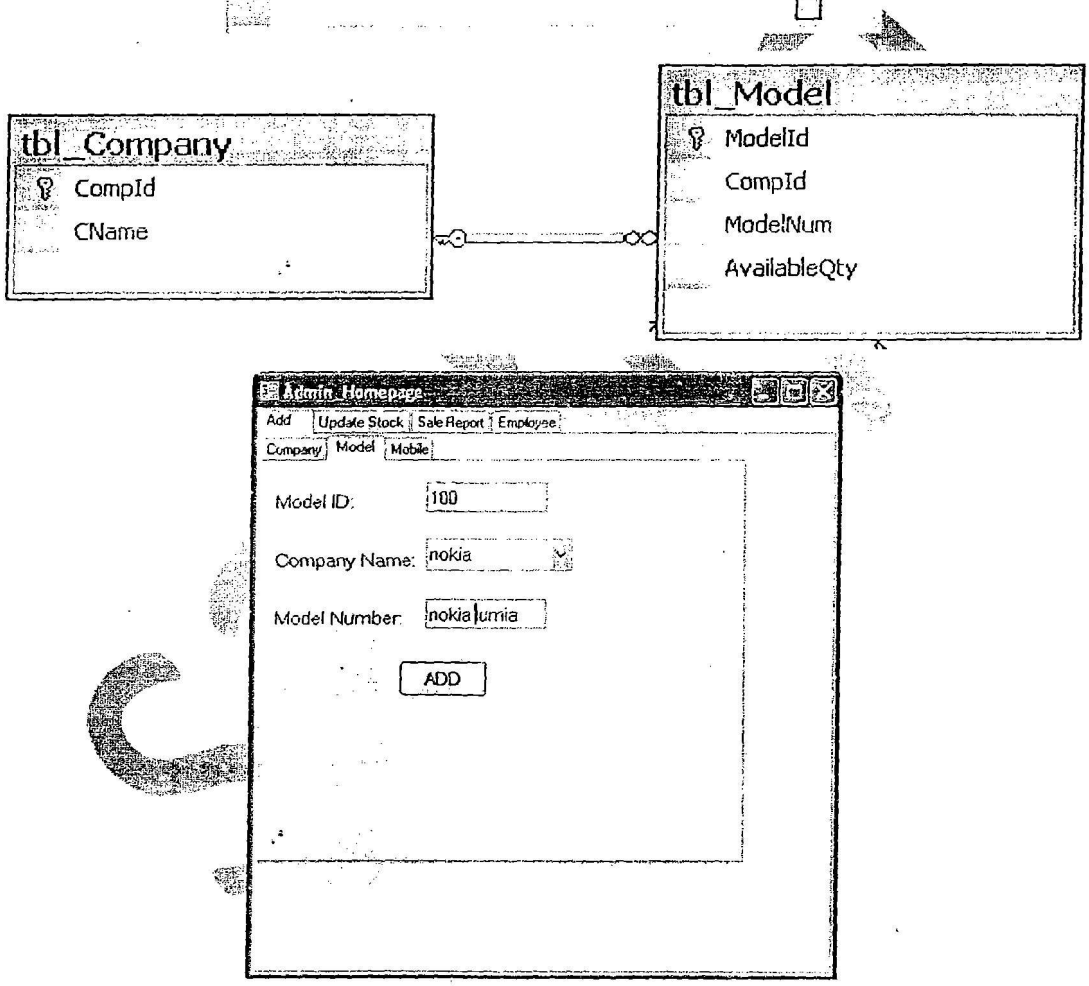
7

Add Model:

Create a table with name "tbl_Model" as show below.

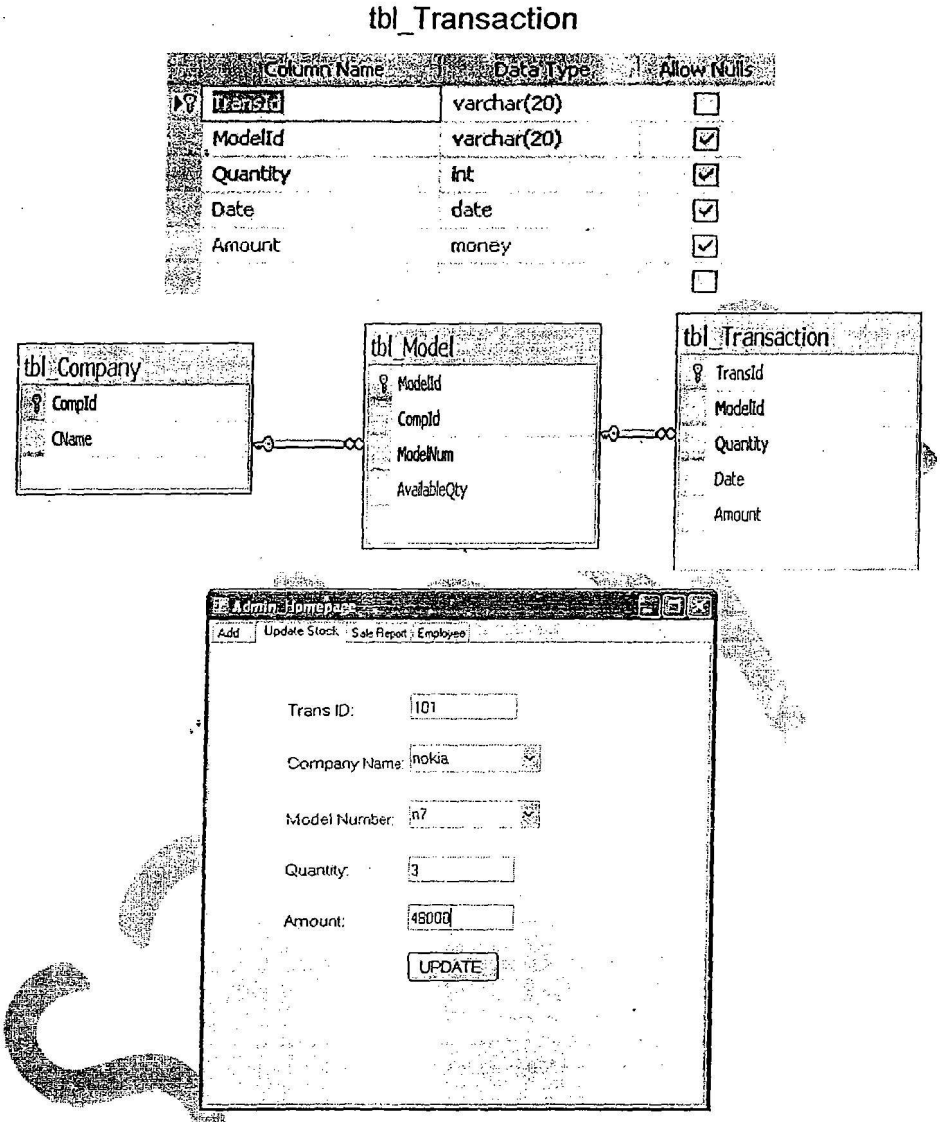
tbl_Model

Column Name	Data Type	Allow Nulls
ModelId	varchar(20)	☐
CompId	varchar(20)	☑
ModelNum	varchar(20)	☑
AvailableQty	int	☑
		☐



- Model Id should be generated automatically from the code and it should **not** be editable.
- Get the company names from "tbl_Company" table.
- When the user clicks on add button the model ID,company ID,model name should be stored in the "tbl_Model" table and quantity should be zero.

Update Stock:

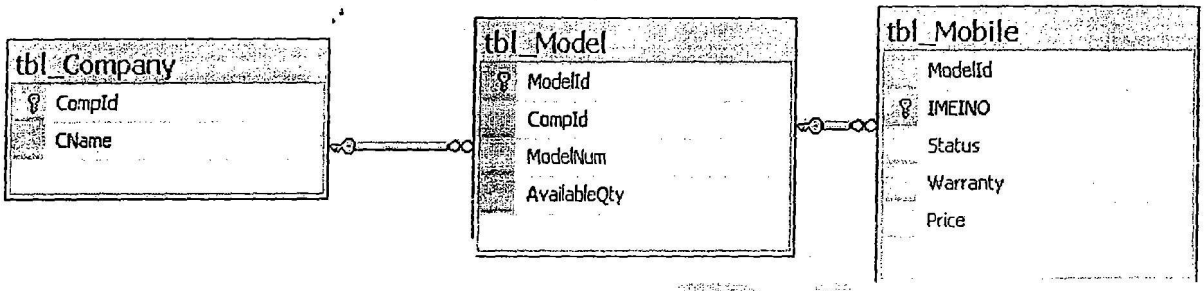


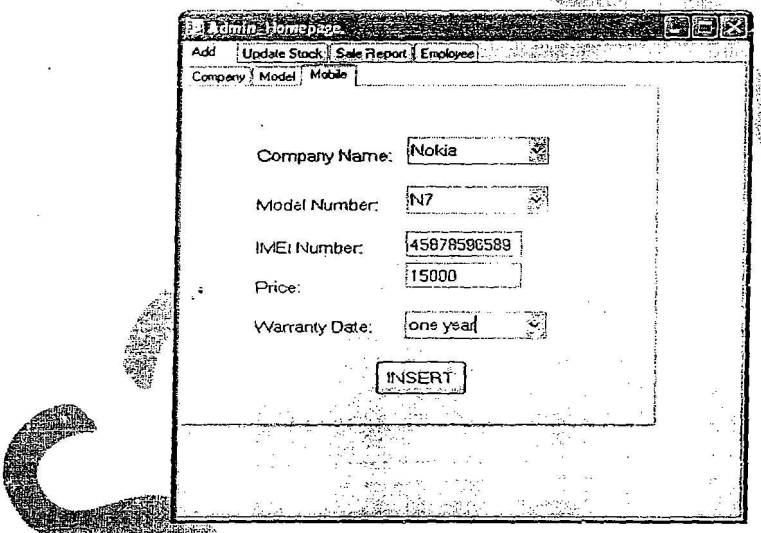
- Trans ID should be generated automatically from the code and it should not be editable.
- Get the company names from "tbl_Company" table and bind in the company name combobox.
- When the user selects company names all the models of that company should be displayed in the model combo box from "tbl_Model" table.
- When the user clicks on update button all transaction details must be stored in "tbl_Transaction" table and 'quantity' must be updated in "tbl_Model" based on modelID.

AddMobile:

tbl_Mobile

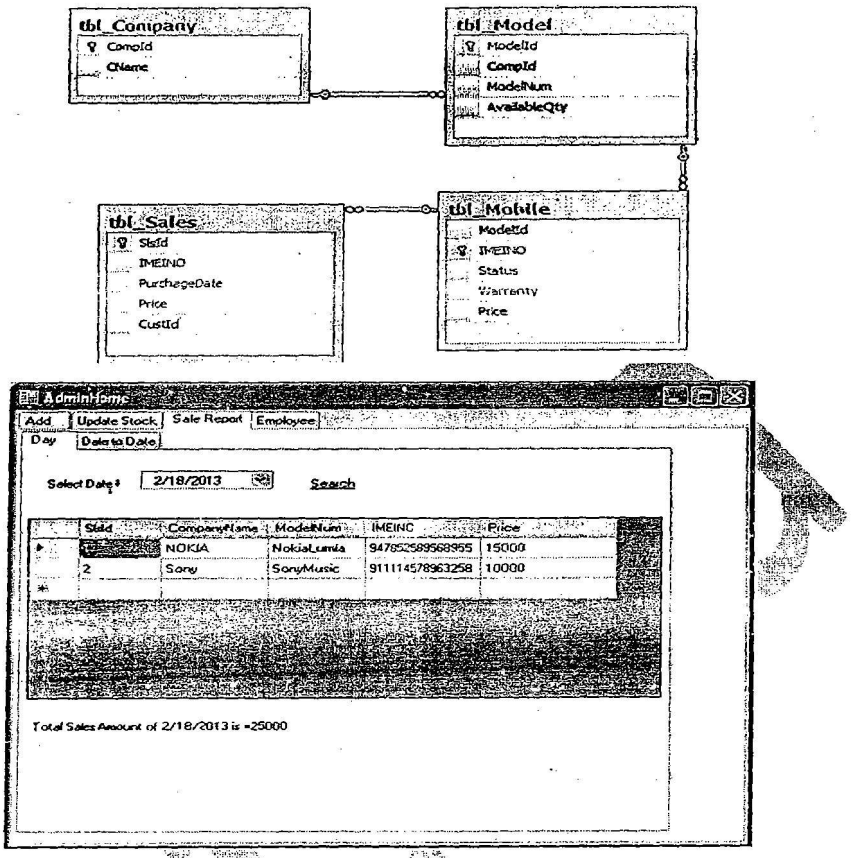
Column Name	Data Type	Allow Nulls
ModelId	varchar(20)	☑
IMEINO	varchar(50)	☐
Status	varchar(20)	☑
Warranty	date	☑
Price	money	☑
		☐





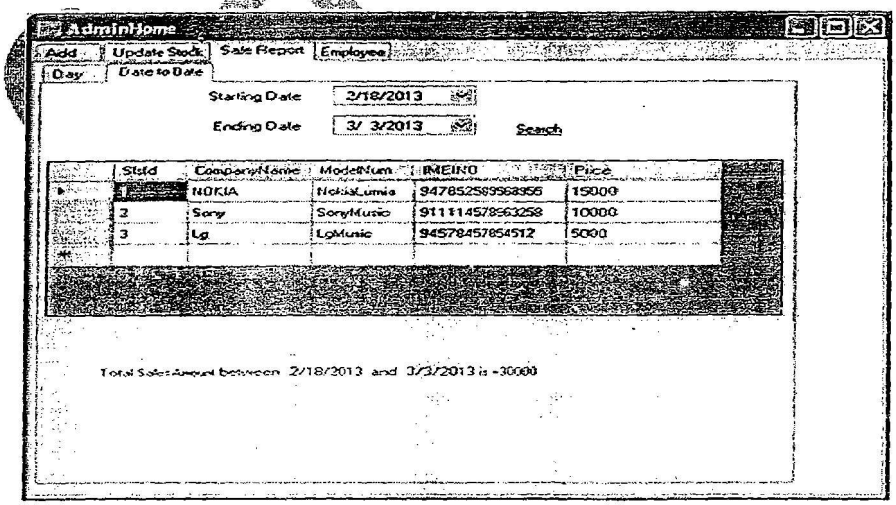
- Get the company name from "tbl_Company" table.
- When the user select company names all the models of that company should be displayed in the model combo box from "tbl_Model" table.
- When the user clicks on insert button modelID, model number, IMEI number, price, warranty date should be inserted in to the "tbl_Mobile" table. By default status must be "Not sold".

Sales Report:



- When user selects the date all sales details of that day should be displayed in the form from three tables displayed above.

Date to Date Report:



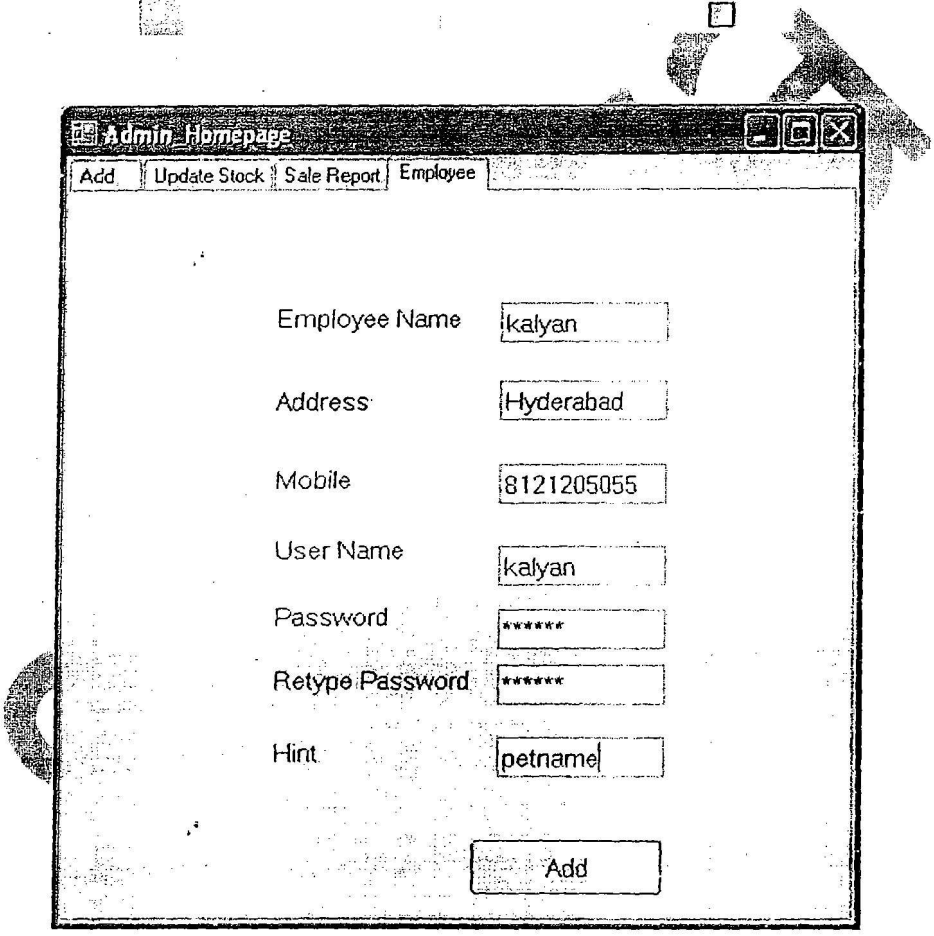
- When user selects the start date and end date all sales details between these days should be displayed in the form from three tables displayed above.
- The total sales amount of the selected dates should be displayed in the bottom of the grid view.

AddEmployee:

Create a table with name "tbl_user".

tbl_User

Column Name	Data Type	Allow Nulls
UserName	varchar(20)	☐
PWD	varchar(20)	☑
EmployeeName	varchar(20)	☑
Address	varchar(MAX)	☑
MobileNUmber	varchar(20)	☑
Hint	varchar(50)	☑
		☐



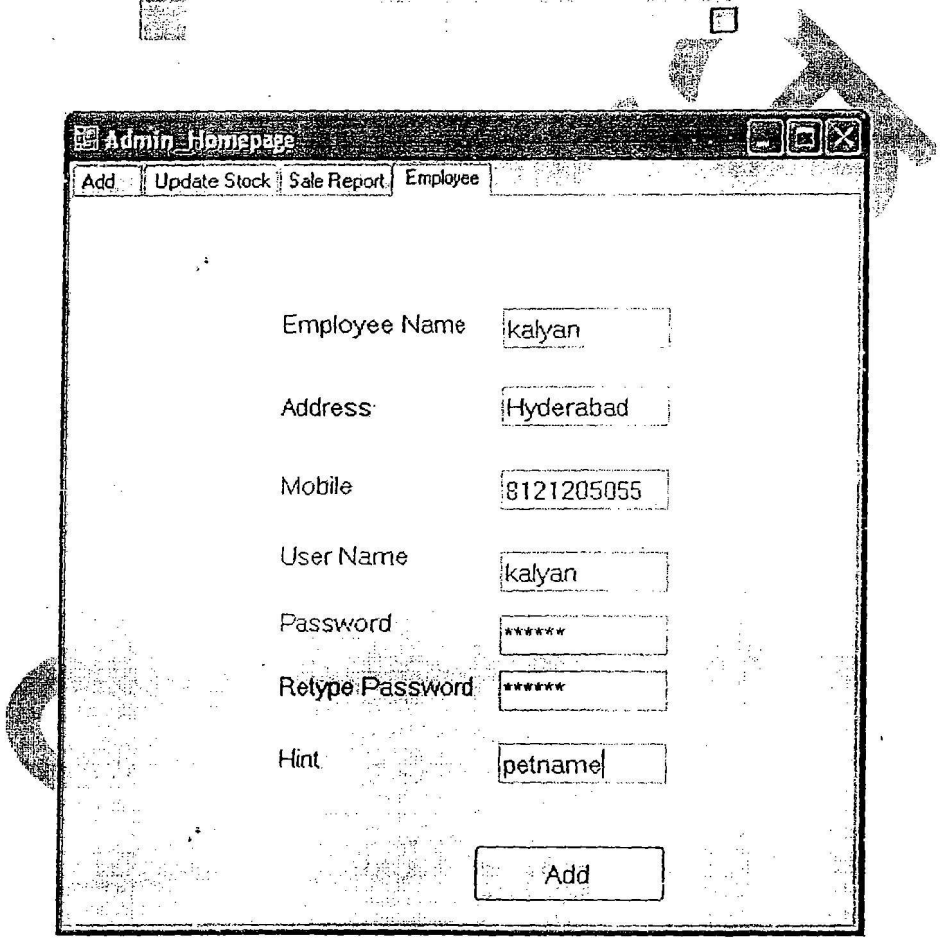
- When user clicks on add button all the details must be stored in "tbl_user" table.

AddEmployee:

Create a table with name "tbl_user".

tbl_User

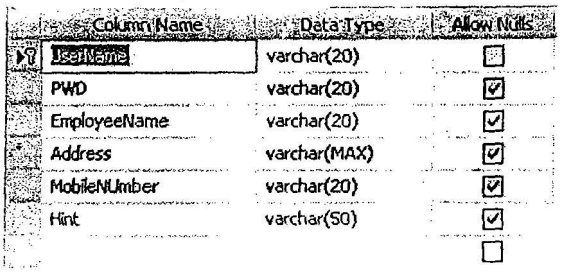
Column Name	Data Type	Allow Nulls
UserName	varchar(20)	☐
PWD	varchar(20)	☑
EmployeeName	varchar(20)	☑
Address	varchar(MAX)	☑
MobileNUmber	varchar(20)	☑
Hint	varchar(50)	☑
		☐



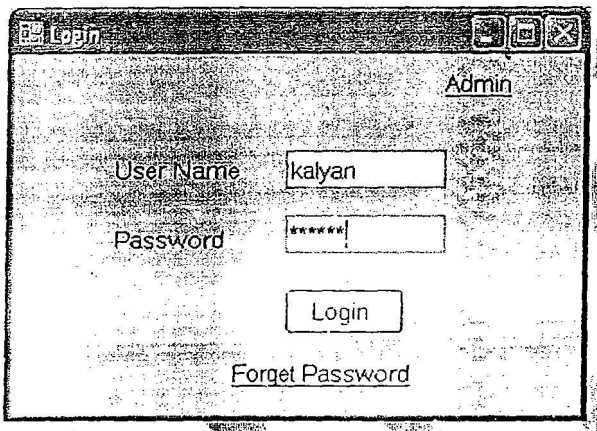
- When user clicks on add button all the details must be stored in "tbl_user" table.

Home Form:

- Create a table with the name "tbl_User".



Column Name	Data Type	Allow Nulls
UserName	varchar(20)	☐
PWD	varchar(20)	☑
EmployeeName	varchar(20)	☑
Address	varchar(MAX)	☑
MobileNUmber	varchar(20)	☑
Hint	varchar(50)	☑
		☐



- The user enter his credentials and clicks on Login button if it is valid redirect to UserHomePage.
- If it is invalid it will display one error message to user.

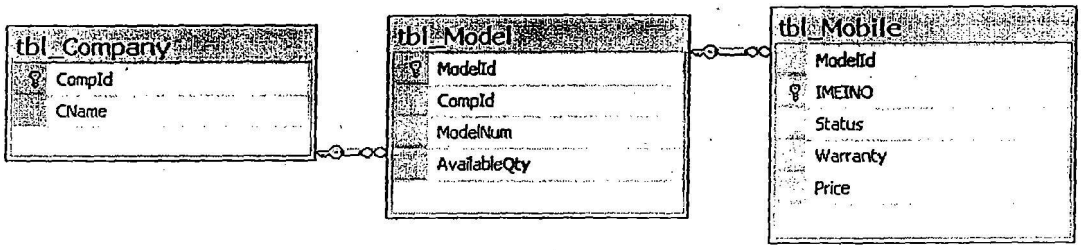
Forgot Password Form:

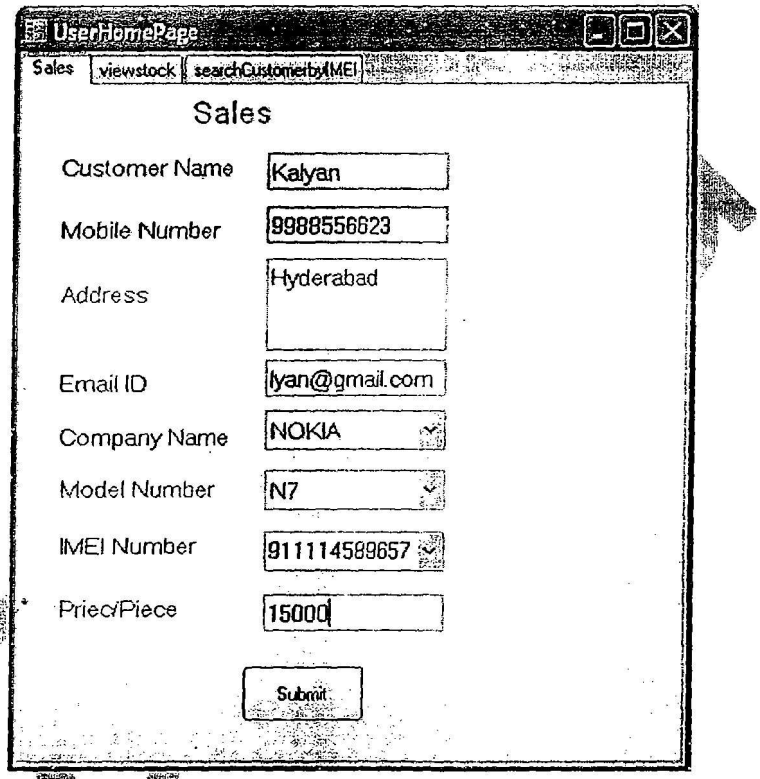
- The User clicks on Forget Password linkbutton, it should redirect to Forget Password Form.

- When the user enter the UserName and Hint, and clicks on the submit button.
- If the UserName and Hint is valid it should display the password in below label, Other wise it display one error message to user.
- Based on UserName and Hint it will get password from "tbl_User" table.

Sales Form:

- Create a table with the name "tbl_Customer".



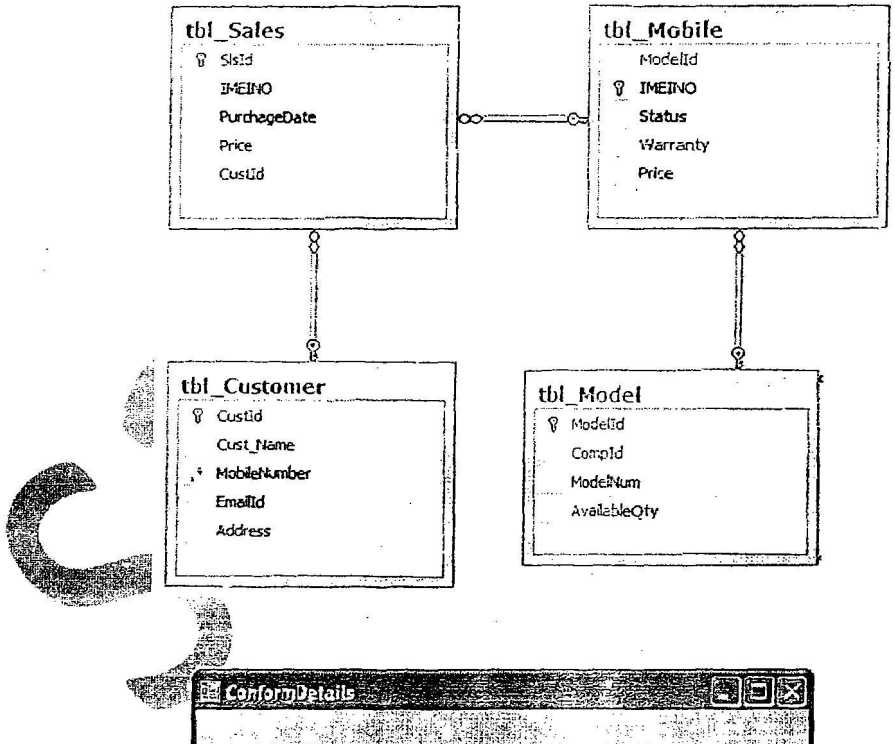


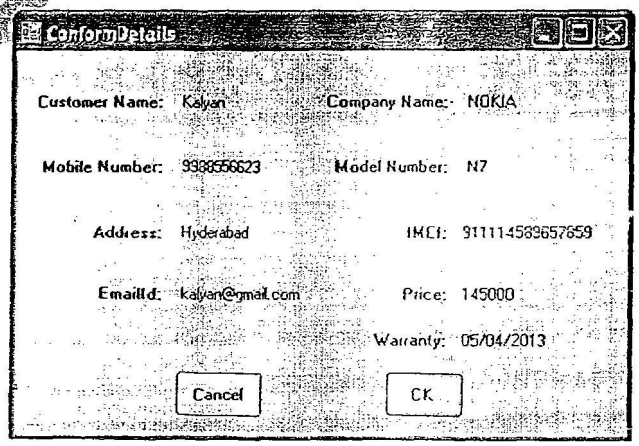
- User enter the details into these fields, Here When the user select the company name in the combo box. The company related models automatically shown in below combo box(Model Number).
- Based on model number that related IMEI numbers also displayed in below (IMEI number)combo box.
- These two combo boxes are should read only.
- When user clicks on submit button it will redirect to conform details Form. and show all the details in labels as shown below.

Column Name	Data Type	Allow Nulls
CustId	varchar(20)	☐
Cust_Name	varchar(20)	☑
MobileNumber	varchar(20)	☑
EmailId	varchar(20)	☑
Address	varchar(MAX)	☑
		☐

DOTNET-PC.MobileS...e - dbo.tbl_Sales

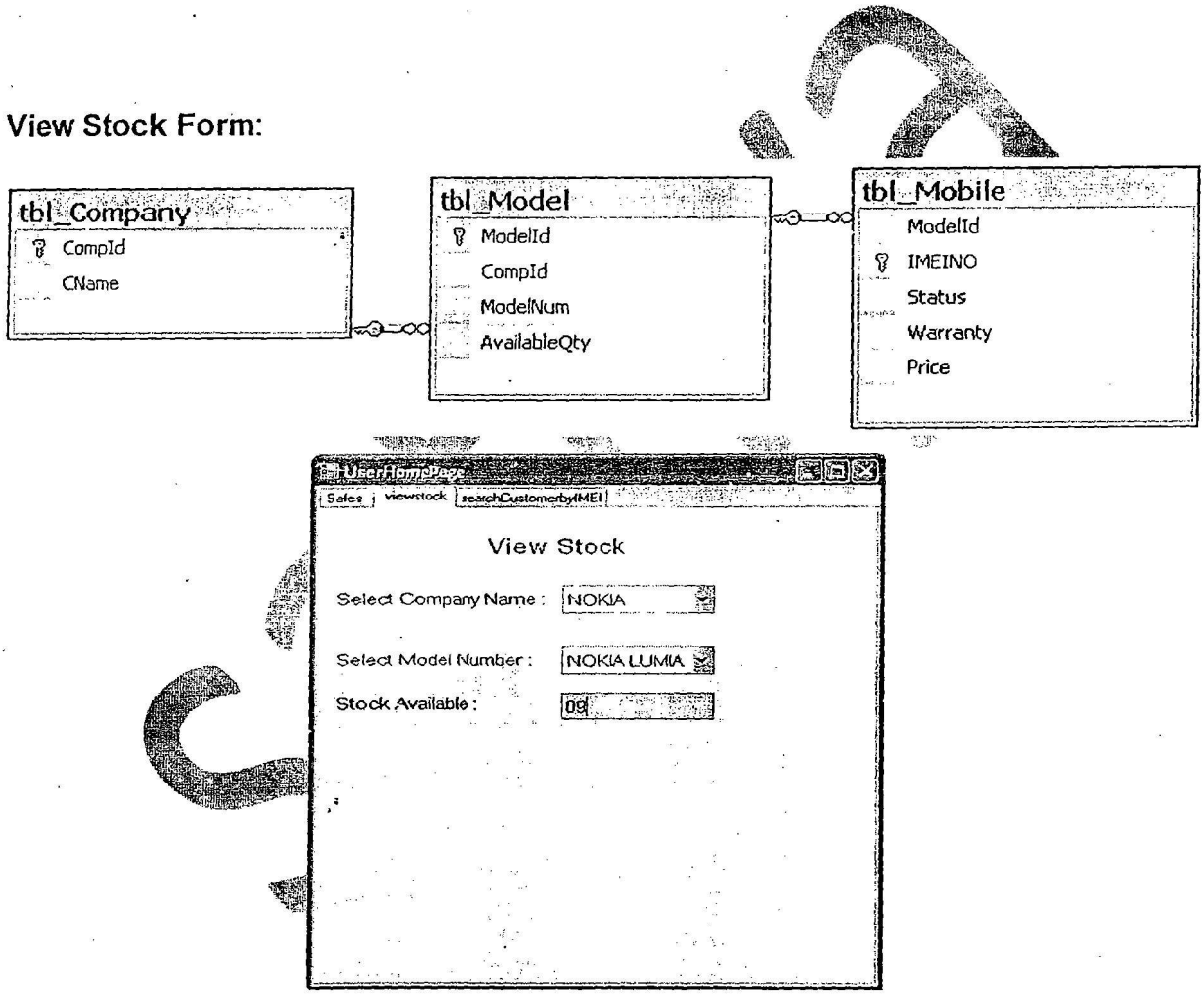
Column Name	Data Type	Allow Nulls
SlsId	varchar(20)	☐
IMEINO	varchar(50)	☑
PurchageDate	date	☑
Price	money	☑
CustId	varchar(20)	☑
		☐





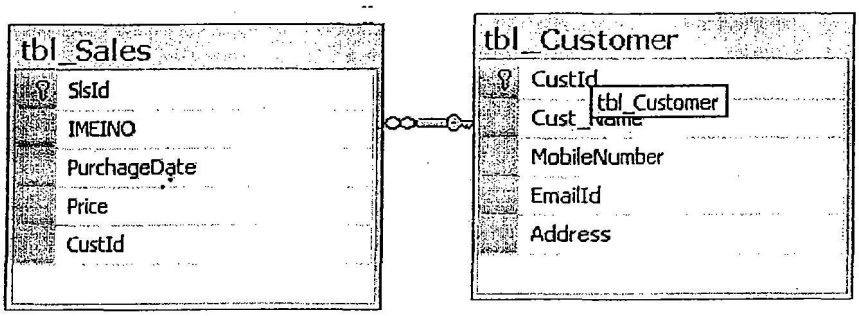
- The sales id and customer id should be generated automatically.
- When the form is loaded based on IMEI number warranty should display
- Here the user clicks on Ok button the Customerid CustomerName, MobileNumber, Address and Emailld should be stored/Inserted in "tbl_Customer" table and Salesid, .IMEI Number, Price and customerid should be inserted into Sales Table
- Based on the IMEI number the status in the mobile table should be changed to "sold"
- The Available Quantity should be updated in model table based on Modelid.
- The user clicks on Cancel button it should not be stored into the database.

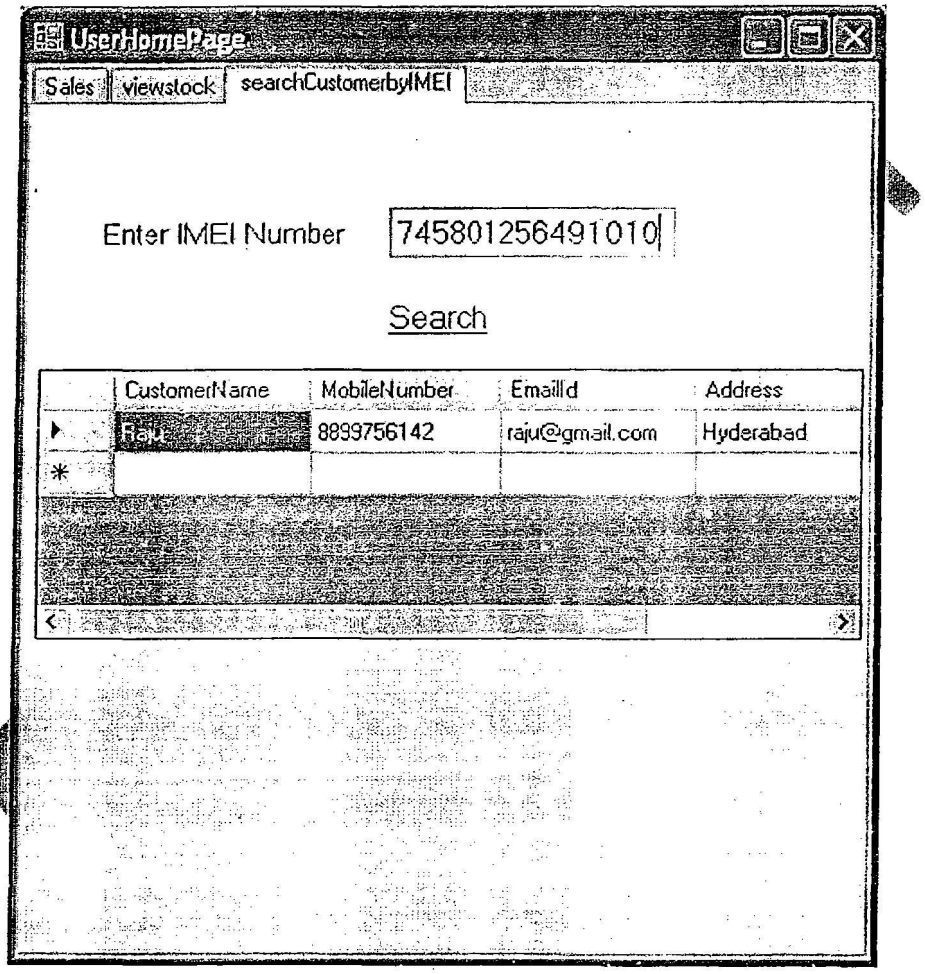
View Stock Form:



- In this Form User can View the Stock Availability.
- When the user select the company name in the combo box. The company related models automatically shown in below combo box.
- Based on the model number it will get the Modelld. Based on Modelld it will show Stock availability from tbl_Model table.
- Here the stock availability textbox should not editable.

Search Customer by IMEI Form:





- Here user can enter the IMEI number and click on submit button.
- If the IMEI number is valid, based on IMEI number it will get the data from "tbl_Sales" and "tbl_Customer" tables for the required fields in dataGridView.
- If the IMEI number is invalid it will display one error message to the User.